Day 3 — Object-Oriented Programming (OOP)

Simple Questions, Answers, Java Examples, and a Student Management Console Application

1. What is Object-Oriented Programming (OOP)?

OOP organizes code around objects. Objects contain data (fields) and behaviour (methods). Java uses classes to define objects. The four main OOP principles are encapsulation, inheritance, polymorphism, and abstraction.

2. What are classes and objects?

A class is a blueprint. An object is an instance of a class.

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println(name + " " + age);
    }
}

// Example inside main():
Student s1 = new Student();
s1.name = "Asha";
s1.age = 20;
s1.display();
```

3. What is a constructor?

A constructor initializes a new object. It has the same name as its class and has no return type.

```
class Student {
    String name;
    int age;

    Student(String studentName, int studentAge) {
        name = studentName;
        age = studentAge;
    }
}
// Example: Student s1 = new Student("Asha", 20);
```

4. What is encapsulation?

Encapsulation protects data by making fields private and providing controlled access through methods such as getters and setters.

```
class Student {
    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

5. What is inheritance?

Inheritance lets one class reuse accessible fields and methods from another class. Java uses the extends keyword.

```
class Person {
    String name;

    void showName() {
        System.out.println(name);
    }
}
class Employee extends Person {
    int employeeId;
}
```

6. What is polymorphism?

Polymorphism means 'many forms'. Method overloading is compile-time polymorphism: methods share a name but have different parameter lists.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Method overriding is another form of polymorphism: a subclass provides its own implementation of an inherited method.

7. What is abstraction?

Abstraction shows essential behaviour while hiding implementation details. Java supports it through abstract classes and interfaces.

```
abstract class Shape {
    abstract double area();
}
class Circle extends Shape {
    double radius = 2;

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}
```

8. What are access modifiers?

Modifier	Basic meaning
public	Accessible broadly, subject to Java module/package rules.
private	Accessible only inside the declaring class.
protected	Accessible in the same package and by subclasses under Java's access rules.
default (no keyword)	Accessible within the same package.

9. Student and Employee practice classes

These examples demonstrate fields, constructors, and methods. For a real project, put public classes in appropriately named .java files.

```
class Student {
    private String name;
    private int rollNo;

    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

class Employee {
    private String name;
    private double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void displayDetails() {
        System.out.println("Employee: " + name);
        System.out.println("Salary: " + salary);
    }
}
```

10. Mini Project: Student Management Console Application

This beginner program takes one student's details and displays them. It demonstrates a class, constructor, private fields, and methods. It does not store data in a database.

```
import java.util.Scanner;

class Student {
    private String name;
    private int rollNo;
    private double marks;

    Student(String name, int rollNo, double marks) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
    }

    void displayDetails() {
        System.out.println("\n--- Student Details ---");
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter student name: ");
        String name = sc.nextLine();

        System.out.print("Enter roll number: ");
        int rollNo = sc.nextInt();

        System.out.print("Enter marks: ");
        double marks = sc.nextDouble();

        Student student = new Student(name, rollNo, marks);
        student.displayDetails();
        sc.close();
    }
}
```

Example output

```
Enter student name: Asha
Enter roll number: 101
Enter marks: 88.5

--- Student Details ---
Name: Asha
Roll No: 101
Marks: 88.5
```

11. Day 3 task checklist

■ Create Student and Employee classes with properties and methods.

■ Practise constructors and private fields with getters/setters.

■ Demonstrate inheritance using Person and Employee.

■ Demonstrate method overloading using a Calculator class.

■ Run the Student Management Console Application.

Quick Revision

Class: blueprint. **Object:** instance of a class. **Constructor:** initializes an object. **Encapsulation:** protects data. **Inheritance:** reuses class behaviour. **Polymorphism:** same method name/interface can have different behaviour. **Abstraction:** exposes essential behaviour while hiding implementation details.